

Ideation Phase

Brainstorm & Idea Prioritization Template

Date	31 January 2025
Team ID	SWTID-2026-1268
Project Name	Script-Controlled ACL – Restrict Record Access Based on Field
Maximum Marks	4 Marks

Brainstorm & Idea Prioritization Template:

Brainstorming provides a free and open environment that encourages everyone within a team to participate in the creative thinking process that leads to problem solving. Prioritizing volume over value, out-of-the-box ideas are welcome and built upon, and all participants are encouraged to collaborate, helping each other develop a rich amount of creative solutions.

Reference: <https://www.mural.co/templates/brainstorm-and-idea-prioritization>

Step-1: Team Gathering, Collaboration and Select the Problem Statement

A. Team gathering

10 minutes

All team members were invited, shared pre-work on ServiceNow ACLs, and agreed on roles: developer, tester, documentation lead and facilitator.

B. Set the goal

Design and implement a **script-controlled ACL** in ServiceNow that restricts access to records (and individual fields) based on the value of a field on the record, the logged-in user's roles, groups and attributes.

C. Define your problem statement

5 minutes

PROBLEM: How might we restrict record access based on a field value (for example Confidentiality, Department, Company or Assignment Group) using a script-controlled ACL, so that only authorized users can read or edit sensitive records without hurting performance or maintainability?

Key rules of brainstorming

Stay in topic. | Encourage wild ideas. | Defer judgment. | Listen to others. | Go for volume. | If possible, be visual.

Step-2: Brainstorm, Idea Listing and Grouping

Brainstorm (10 minutes) – ideas that address the problem statement. Each sticky note is one idea from the team.

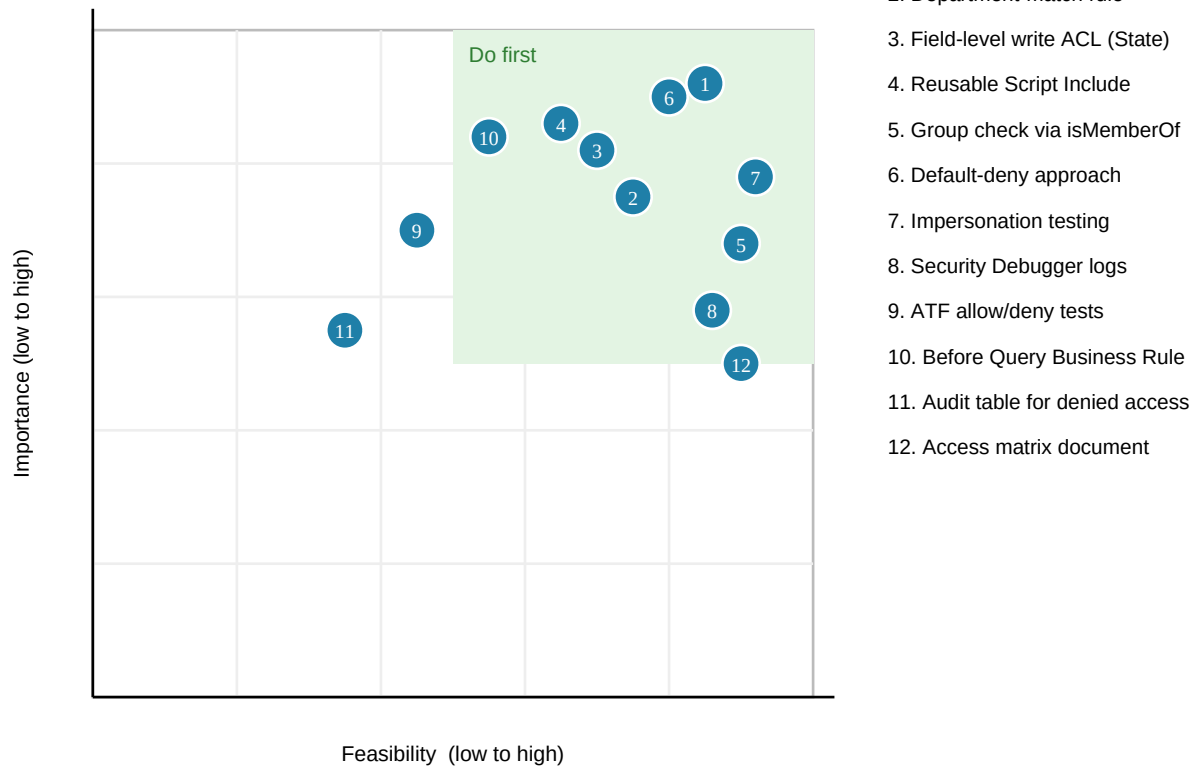
Read ACL on Incident: allow only if current.confidential == false or user has the itil_admin role	Reusable Script Include (ACLHelper) holding all access logic, called from the ACL script	Test every rule by impersonating users from different roles and groups	Add a Before Query Business Rule so lists also hide restricted rows
Restrict by caller's department: compare current.caller_id.department with gs.getUser().department	Check group membership with gs.getUser().isMemberOf(current.assignment_group)	Use Security Debugger / ACL debug logs to see which rule allowed or denied	Log denied access attempts to a custom audit table
Field-level write ACL so only assigned_to can edit the State field	Default-deny approach: set answer = false first, then grant only on matching conditions	Automated Test Framework (ATF) cases for allow and deny scenarios	Access matrix document: role x field value x operation (read / write / create / delete)

Group ideas (20 minutes) – similar ideas clustered, each cluster given a sentence-like label.

Cluster 1: Define access rules based on field values	Confidential flag check; department match; field-level write restriction; default-deny approach.
Cluster 2: Write clean, reusable ACL scripts	Script Include (ACLHelper); group-membership checks with isMemberOf; keep each script short and commented.
Cluster 3: Prove it works before release	Impersonation testing; Security Debugger logs; ATF allow/deny test cases.
Cluster 4: Keep it fast, visible and auditable	Before Query Business Rule for lists; audit log of denied access; access matrix document.

Step-3: Idea Prioritization

Prioritize (20 minutes) – ideas placed on the grid by **Importance** (positive impact if done without difficulty or cost) and **Feasibility** (cost, time, effort, complexity).



Prioritization outcome

Priority	Ideas selected to build first
High (important and feasible)	1. Confidential-flag read ACL, 6. Default-deny approach, 3. Field-level write ACL, 4. Reusable Script Include, 7. Impersonation testing
Next (important, more effort)	10. Before Query Business Rule for lists, 9. ATF allow/deny tests
Later (nice to have)	2. Department-match rule, 5. isMemberOf checks, 8. Debugger logs, 11. Audit table, 12. Access matrix